Lista de Exercícios — .NET 10, C# e JWT

Tema: APIs, autenticação, autorização e proteção de rotas

Regras de entrega

Individual, codigo fonte no git e data de entrega de acordo com AVA

Leia os enunciados com atenção. Nas questões teóricas, responda com suas próprias palavras. No exercício prático, implemente e teste a API conforme os requisitos apresentados.

Parte 1 — Exercícios teóricos

Exercício 1 — Autenticação e autorização

Explique, com suas palavras, a diferença entre autenticação e autorização.

Apresente um exemplo de uma situação em que um usuário precisa ser autenticado para acessar uma API e outro exemplo em que, além de estar autenticado, ele precisa ter uma permissão específica para realizar uma operação.

A autenticação é para provar a identidade do usuário. O sistema valida a identidade, por login, por exemplo apresentar o RG ou crachá em uma portaria. Autorização é conferir o que o usuário pode fazer, como executar ação ou acessar recursos.

Exemplo de Apenas Autenticação: Um usuário comum acessa um aplicativo de e-commerce e precisa se autenticar, com o login, para conseguir visualizar o histórico dos seus próprios pedidos na rota `GET /api/meus-pedidos`.

Exemplo de Autenticação + Permissão específica:O mesmo usuário tenta acessar a rota `DELETE /api/produtos/123` para apagar um produto do catálogo da loja. Mesmo estando logado (autenticado), a API vai bloquear a operação porque ele não possui o cargo/permissão de **"Administrador"** (autorização) para alterar o estoque.

Exercício 2 — Funcionamento do JWT

Descreva o fluxo de autenticação utilizando JWT, desde o envio do usuário e da senha até o acesso a uma rota protegida.

- **O fluxo de autenticação passo a passo:**
    1. O cliente (Postman, aplicativo ou site) manda o usuário e a senha em formato JSON através de uma requisição `POST /api/auth/login`.

2. A API valida as credenciais. Se estiverem corretas, ela gera um token JWT criptografado/assinado usando uma chave secreta interna e o devolve no corpo da resposta (`200 OK`).
3. O cliente recebe esse token e o armazena localmente (em memória ou no navegador).
4. Para acessar qualquer rota protegida (como `/api/usuario/perfil`), o cliente anexa esse token no cabeçalho de todas as próximas requisições HTTP.
5. A API recebe a requisição da rota protegida, descriptografa a assinatura do token usando a mesma chave secreta para checar se ele é válido/legítimo e, se estiver tudo certo, libera o acesso ao recurso.

Em sua resposta, explique:

● Qual é a finalidade do token JWT?
Ele é uma credencial de acesso temporária e segura. A API não consulta o banco a cada segundo para identificar o usuário, porque todas as informações necessárias já vem compactadas no token

● Como o cliente envia o token nas próximas requisições?
Dentro dos cabeçalhos ocultos dentro da requisição HTTP.

● Para que serve o cabeçalho HTTP `Authorization: Bearer`?
Avisa formalmente para o servidor que a requisição está mandando uma credencial de autenticação. A palavra Bearer mostra para o framework que qualquer uma que esteja carregando o token anexado tem o direito de acessar o recurso solicitado.

● Por que não devemos colocar senhas ou informações secretas no conteúdo do token?
O JWT é só um texto em Base64. Ele não é criptografado. Qualquer um que interceptar o token pode copiar, acessar o site como jwt.io e ler tudo que está nele. Ele só carrega identificação pública, nunca dados sensíveis.

Exercício 3 — Status HTTP 401 e 403

Explique a diferença entre os códigos HTTP **401 Unauthorized** e **403 Forbidden**.

Apresente uma situação prática para cada código, considerando uma API que utiliza JWT para proteger suas rotas.

A diferença entre os códigos:
401 Unauthorized (Não Autenticado): Significa que o servidor não sabe quem você é. A requisição falhou porque o token JWT não foi enviado, expirou ou está completamente inválido. O sistema exige que você faça login primeiro.

403 Forbidden (Proibido/Não Autorizado): Significa que o servidor sabe exatamente quem você é, mas você não tem o direito de acessar esse recurso. A sua identidade foi validada, mas o seu nível de privilégio (Role/Permissão) é insuficiente.
Situações práticas em uma API com JWT:
Caso Prático do 401 Unauthorized: Você tenta acessar a rota protegida `GET /api/usuario/perfil`, mas esquece de ativar a aba de autenticação no Postman (ou envia um token que expirou há duas horas). A API recusa o acesso de imediato com o código 401.
Caso Prático do 403 Forbidden: Você faz o login válido com um usuário que possui a regra comum de `"Cliente"`. Você pega o token JWT gerado e tenta enviá-lo na rota `GET /api/admin/relatorio-financeiro`. A API lê o seu token, valida que você é o "João", mas responde com o código 403 porque a rota exige o nível `"Financeiro"` ou `"Diretoria"`.

Parte 2 — Exercício prático

Exercício 4 — Desenvolvendo uma API com autenticação JWT

Utilizando C#, ASP.NET Core e Visual Studio, desenvolva uma API que implemente autenticação com JWT.

Não é necessário utilizar banco de dados. Para este exercício, utilize um usuário e uma senha fixos no código, exclusivamente para fins didáticos.

Requisitos obrigatórios

1. Criar um projeto do tipo ASP.NET Core Web API.
2. Criar a rota `POST /api/auth/login`, que receba usuário e senha em formato JSON.
3. Validar as credenciais informadas utilizando os valores fixos definidos no código.
4. Quando as credenciais estiverem corretas, gerar e retornar um token JWT. 5. Quando as credenciais estiverem incorretas, retornar o status HTTP `401 Unauthorized`.
6. Criar uma rota protegida, por exemplo, `GET /api/usuario/perfil`, utilizando o atributo `[Authorize]`.
7. Fazer com que a rota protegida retorne uma mensagem contendo o nome do usuário autenticado.
8. Testar todas as funcionalidades no Postman.

Testes obrigatórios no Postman

Realize os seguintes testes e confira os resultados:

Teste 1 — Login válido

Envie um usuário e uma senha corretos. A API deverá retornar `200 OK` e um token

JWT. **Teste 2 — Login inválido**

Envie um usuário ou uma senha incorretos. A API deverá retornar `401`

`Unauthorized`. **Teste 3 — Acesso sem token**

Acesse a rota protegida sem enviar um token. A API deverá retornar `401 Unauthorized`.

Teste 4 — Acesso com token válido

Envie o token obtido no login por meio da opção **Authorization** → **Bearer Token** do Postman. A API deverá retornar `200 OK` e os dados ou a mensagem definidos na implementação.

O que deve ser entregue

- Código-fonte completo em um repositório Git.
- Arquivo README com instruções para executar o projeto e realizar os testes. ● Evidências dos testes no Postman, por meio de capturas de tela ou de uma coleção exportada.
- Código organizado e com nomes claros para controllers, métodos e classes.

Observação: não inclua chaves secretas reais ou senhas pessoais no repositório.

Bom trabalho! O objetivo é compreender o funcionamento da autenticação JWT e saber implementar e testar uma API protegida.